Projeto final: plataforma de delivery

Uma aplicação Laravel completa construída em três dias de treinamento

Arquitetura, dados, frontend, integrações, qualidade e operação em um único produto

Resultado da imersão

01	**Produto funcional**	Cliente pede, restaurante aceita e entregador atualiza a rota
02	**Arquitetura sustentável**	Casos de uso explícitos, contratos próprios, eventos e jobs
03	**Qualidade demonstrável**	Testes, segurança, observabilidade e métricas operacionais
04	**Operação segura**	GitHub Actions, deploy reproduzível e rollback verificável

Escopo funcional do DeliveryHub

O sistema coordena pedidos de vários restaurantes sem misturar dados, estoque ou decisões operacionais.

ATORES

- cliente cria e acompanha pedidos
- restaurante gerencia catálogo e preparo
- entregador aceita rota e atualiza posição
- operador audita falhas e reprocessa entregas

FLUXO

- menu e carrinho
- checkout e pagamento
- preparo e despacho
- rastreamento e entrega

Arquitetura de referência

O projeto começa como monólito modular e separa capacidades sem criar serviços distribuídos antes da necessidade.

Domínio

- módulos por capacidade
- estados válidos no agregado
- regras em Actions e Value Objects

Aplicação

- contratos no container
- DTOs nas fronteiras
- events, listeners e jobs

Infraestrutura

- MySQL e Redis
- Reverb e HTTP clients
- logs, métricas e deploy

A separação segue o risco do negócio e mantém cada mudança localizada e testável.

Módulos e contratos

O Service Container liga contratos do domínio a adapters de infraestrutura sem contaminar os casos de uso.

```
public function register(): void
{
    $this->app->bind(
        PaymentGateway::class,
        StripePaymentGateway::class,
    );

    $this->app->bind(
        RoutePlanner::class,
        MapboxRoutePlanner::class,
    );
}
```

Fronteiras

- Catalog
- Ordering
- Payments
- Delivery

Controllers recebem HTTP. Actions coordenam casos de uso. Adapters encapsulam serviços externos.

Plano de 24 horas

Dia 1 · Núcleo

Modelar catálogo, carrinho e pedido. Fechar o checkout com consistência e resposta de API previsível.

- arquitetura modular
- Eloquent e concorrência
- React e API

Dias 2 e 3 · Produção

Conectar pagamento, filas e tempo real. Encerrar com segurança, observabilidade e deploy.

- eventos e integrações
- testes e proteção
- CI/CD e operação

Cada dia termina com uma entrega executável, testes verdes e uma demonstração curta do fluxo.

Estados do pedido

O agregado Order aceita somente transições válidas e mantém o fluxo previsível para API, jobs e interface.

```
enum OrderStatus: string
{
    case Pending = 'pending';
    case Paid = 'paid';
    case Preparing = 'preparing';
    case Ready = 'ready';
    case Delivering = 'delivering';
    case Delivered = 'delivered';
    case Cancelled = 'cancelled';
}
```

Invariantes

- pagamento antes do preparo
- uma entrega ativa
- delivered é terminal
- cancelamento por regra

As transições pertencem ao domínio. Controllers e listeners apenas solicitam a mudança.

Checkout como caso de uso

A Action recebe um DTO validado, abre a transação e retorna um pedido consistente sem conhecer HTTP.

```
final class Checkout
{
    public function execute(
        User $customer,
        CheckoutData $data,
    ): Order {
        return DB::transaction(fn () =>
            Order::createFromCart($customer, $data)
        );
    }
}
```

Fronteira

- Form Request fora
- DTO imutável
- transação curta
- evento após commit

O caso de uso concentra a regra e permite testes sem depender do controller.

Modelo de dados e consultas

Relacionamentos explícitos e índices guiados pelas consultas evitam N+1 e preservam o tenant em cada acesso.

```
final class Order extends Model
{
    public function restaurant(): BelongsTo
    {
        return $this->belongsTo(Restaurant::class);
    }

    public function items(): HasMany
    {
        return $this->hasMany(OrderItem::class);
    }
}
```

Banco

- tenant_id e status
- índice por tenant e status
- external_id único
- valores em centavos

Carregue somente relações necessárias e confirme o plano das consultas críticas.

Reserva de estoque concorrente

O checkout bloqueia somente a linha de estoque necessária e impede que dois pedidos consumam a última unidade.

```
return DB::transaction(function () use ($item, $qty) {
    $stock = Stock::query()
        ->where('menu_item_id', $item->id)
        ->lockForUpdate()
        ->firstOrFail();

    throw_if($stock->available < $qty, OutOfStock::class);
    $stock->decrement('available', $qty);

    return $stock;
});
```

Consistência

- lockForUpdate
- transação curta
- retry de deadlock
- teste concorrente

O lock protege a unidade de disputa; a transação curta preserva throughput e reduz deadlocks.

API de pedidos eficiente

A listagem usa escopo de tenant, eager loading seletivo e cursor pagination para manter latência e memória previsíveis.

```
$orders = Order::query()
    ->forTenant($tenantId)
    ->with([
        'restaurant:id,name',
        'items:id,order_id,name,quantity',
    ])
    ->latest('id')
    ->cursorPaginate(50);

return OrderResource::collection($orders);
```

Contrato

- API Resource
- cursor pagination
- eager loading
- campos estáveis

O contrato permanece estável enquanto a consulta traz apenas os dados necessários para a tela.

Checkout com React e Inertia

O formulário mantém estado, exibe erros 422 e bloqueia reenvio enquanto o checkout está em processamento.

```
const form = useForm<CheckoutPayload>({
    address_id: '',
    payment_method: 'card',
});

function submit() {
    form.post('/checkout', {
        preserveScroll: true,
    });
}

return <CheckoutForm form={form} onSubmit={submit} />;
```

Interface

- erros por campo
- estado de loading
- erro 422 contextual
- feedback previsível

O frontend melhora a experiência, mas o backend continua responsável por autorização e invariantes.

Teste do checkout

O teste verifica resposta, persistência, estoque e evento sem acoplar a expectativa à implementação interna.

```
it('creates a paid order without overselling', function () {
    PaymentGateway::fake()->approve();

    $response = $this->actingAs($customer)->postJson(
        '/api/orders',
        CheckoutPayload::valid(),
    );

    $response->assertCreated()
        ->assertJsonPath('data.status', 'paid');
    expect(Stock::first()->available)->toBe(9);
    Event::assertDispatched(OrderPaid::class);
});
```

Cenários

- estoque esgotado
-
-
- requisição duplicada
- item indisponível ou tenant incorreto

O teste protege o comportamento observável: contrato HTTP, dados persistidos e efeitos assíncronos.

Evento após o commit

O evento nasce somente depois do commit e carrega um contrato pequeno, estável e serializável.

```
final readonly class OrderPaid
    implements ShouldDispatchAfterCommit
{
    use Dispatchable;

    public function __construct(
        public int $orderId,
        public int $tenantId,
        public CarbonImmutable $occurredAt,
    ) {}
}
```

Payload

- IDs estáveis
- tenant explícito
- occurred_at
- versão do contrato

O evento registra o fato. Cada listener escolhe como reagir sem ampliar a transação do checkout.

Job idempotente para o restaurante

O job persiste a tentativa antes da integração e pode ser executado novamente sem notificar o restaurante duas vezes.

```
public function handle(RestaurantGateway $gateway): void
{
    $attempt = DeliveryAttempt::firstOrCreate([
        'event_id' => $this->eventId,
    ]);

    if ($attempt->sent_at) return;

    $response = $gateway->send(
        $this->orderId,
        $this->eventId,
    );

    $attempt->markSent($response->externalId);
}
```

Execução

- tries e backoff
- timeout curto
- falha observável
- idempotência persistida

A fila reduz latência percebida. Idempotência e persistência tornam o retry seguro.

Webhook de **pagamento** autenticado

O endpoint valida a assinatura sobre o corpo bruto, deduplica o evento do provedor e delega o processamento à fila.

```
$expected = hash_hmac(
    'sha256',
    $request->getContent(),
    config('payments.webhook_secret'),
);

abort_unless(hash_equals(
    $expected,
    $request->header('X-Signature'),
), 401);

ProcessPaymentWebhook::dispatch(
    $request->string('id'),
);
```

Proteção

- corpo bruto
- timestamp e replay
- event_id único
- processamento assíncrono

O webhook responde rápido. O job aplica a transição somente uma vez e registra o resultado.

Outbox transacional

O pedido e a intenção de publicar são persistidos na mesma transação para evitar perda silenciosa de eventos.

```php
DB::transaction(function () use ($data) {
    $order = Order::createFromCart($data);

    OutboxMessage::create([
        'event_id' => (string) Str::uuid(),
        'type' => 'order.created.v1',
        'tenant_id' => $order->tenant_id,
        'payload' => [
            'order_id' => $order->id,
        ],
    ]);
});
```

Publisher

- claim com lock
- entrega idempotente
- métricas de backlog
- retenção e replay

Outbox custa operação adicional, mas protege fluxos em que perder um evento é inaceitável.

Redis para cache e exclusão mútua

Menu usa cache versionado. Checkout usa lock curto apenas para impedir reenvio simultâneo da mesma operação.

```
$menu = Cache::tags(["restaurant:$restaurantId"])
    ->remember(
        "menu:v2:$restaurantId",
        now()->addMinutes(10),
        fn () => MenuQuery::for($restaurantId),
    );

return Cache::lock("checkout:$key", 10)
    ->block(3, fn () => $checkout->execute($data));
```

Política

- chave com versão
- invalidação por tag
- TTL como proteção
- lock com prazo

Cache acelera leitura; locks têm prazo curto e nunca substituem a consistência do banco.

Rastreamento em tempo real

O entregador publica posições com frequência controlada e cada cliente escuta apenas o canal privado do próprio pedido.

```
final class DeliveryLocationUpdated
    implements ShouldBroadcast
{
    use Dispatchable;

    public function broadcastOn(): array
    {
        return [new PrivateChannel(
            "orders.{$this->orderId}"
        )];
    }

    public function broadcastAs(): string
    {
        return 'delivery.location.updated';
    }
}
```

Canal

- autorização privada
- payload mínimo
- throttle de posição
- reconexão no cliente

O banco guarda checkpoints relevantes. O websocket distribui atualizações efêmeras com baixa latência.

Teste do fluxo assíncrono

O teste isola integrações, executa o caso de uso e comprova os efeitos que fazem parte do contrato do produto.

```
it('dispatches fulfillment after payment', function () {
    Queue::fake();
    Event::fake([OrderPaid::class]);

    $order = Order::factory()->pending()->create();
    app(ConfirmPayment::class)->execute(
        order: $order,
        providerEventId: 'evt_123',
    );

    expect($order->refresh()->status)
        ->toBe(OrderStatus::Paid);
    Event::assertDispatched(OrderPaid::class);
});
```

Evidência

- estado persistido
- evento emitido
- job enfileirado
- retry sem duplicação

O teste prova a mudança de estado e os efeitos assíncronos sem depender de workers ou serviços externos.

Autorização multi-tenant

Toda operação valida o papel do usuário e confirma que pedido, restaurante e ator pertencem ao mesmo tenant.

```
public function view(User $user, Order $order): bool
{
    return $user->tenant_id === $order->tenant_id
        && ($user->isOperator()
            || $order->customer_id === $user->id);
}

public function updateStatus(User $user, Order $order): bool
{
    return $user->tenant_id === $order->tenant_id
        && $user->canManage($order->restaurant);
}
```

Proteção

- Policy no backend
- scopes por tenant
- rate limit por ator
- auditoria de decisão

Filtros na interface ajudam a experiência. A autorização definitiva permanece no backend.

Observabilidade do pedido

Cada requisição deve ser rastreável do checkout aos jobs, webhooks e broadcasts que completam o fluxo.

Correlação

- request_id
- event_id
- order_id e tenant_id

Sinais

- p95 do checkout
- idade da fila
- falhas por integração

Diagnóstico

- logs estruturados
- Pulse e métricas
- replay auditável

Um alerta útil aponta impacto, contexto e o procedimento seguro de recuperação.

Pipeline no GitHub Actions

O pipeline bloqueia regressões, produz um artefato versionado e promove a mesma versão entre ambientes.

Qualidade

- composer validate
- Pint e análise estática
- testes com serviços

Build

- assets de produção
- imagem imutável
- migrations verificadas

Entrega

- smoke em staging
- aprovação de produção
- rollback da release

Proteção de branch exige pipeline verde e revisão antes do merge.

Plano de trabalho e checkpoints

Cada dia combina explicação curta, implementação guiada, trabalho em equipe e revisão baseada em evidências.

Dia 1 · Núcleo

- domínio e migrations
- checkout, API, React e testes

Dia 2 · Fluxos

- pagamento e outbox
-
- falhas e reprocessamento

Dia 3 · Produção

- segurança e auditoria
- observabilidade e CI/CD
- deploy e incidente

Checkpoint diário: cenário demonstrado, testes verdes, logs legíveis e decisão técnica explicada.

Entrega final da equipe

Entregar um sistema de delivery executável, testado e operável, com decisões técnicas explicadas por evidências.

Entregas

- repositório reproduzível
- pipeline verde
- TESTING.md da demonstração
- runbook de operação

Critérios de aceite

- fluxo principal completo
- falha e recuperação
- logs e métricas úteis
- deploy e rollback

A apresentação mostra o problema, a decisão, o teste e o comportamento observado em execução.

Checklist final do projeto

1	**Modelar**	módulos, estados e dados
2	**Proteger**	tenant, autorização e concorrência
3	**Integrar**	filas, pagamentos e realtime
4	**Verificar**	testes, performance e observabilidade
5	**Entregar**	pipeline, deploy e rollback

Resultado esperado: uma plataforma de delivery executável, explicável e recuperável diante de falhas.